

Project Proposal Document

Neural Recall

A Memory Game Mobile Application Built with Flutter

Course Name: Mobile Computing

Course Code: CMPE 408

Prepared By:

Anas Alsakkar

Bara Amro

Abdallah Ali

Presentation Date: 7 April

April 6, 2026

Contents

1	Project Description	2
2	Project Objectives	2
3	Game Logic Overview	3
4	Additional Game Features	3
4.1	Leaderboard System	3
4.2	Streak System	4
5	Detailed Analysis Using Wireframes	5
5.1	Screen 1 — Main Menu	5
5.2	Screen 2 — Focus Mode Game Board (2×2)	6
5.3	Screen 3 — Overdrive Mode Game Board (3×3)	7
5.4	Screen 4 — Game Over Overlay	9
6	Flutter Widgets and Functional Implementation	10
6.1	Application-Level Widgets	10
6.2	Main Menu Screen Widgets	10
6.3	Focus Mode and Overdrive Mode Widgets	11
6.4	Leaderboard Widgets	11
6.5	Game Over Overlay Widgets	11
6.6	Functional Modules	12
7	System Architecture and Code Structure	12
8	Semester Deliverables and Progress Presentation Plan	13
8.1	Detailed Semester Deliverables	14
9	Final Acceptance Criteria	15
10	User Acceptance Test Cases	16
11	Risk Assessment and Mitigation	17
12	Conclusion	17

1. Project Description

Neural Recall is a mobile memory game developed with Flutter. The application is designed to test and improve short-term memory, reaction speed, and sequential recall through a clean and modern cyber-themed interface. The project emphasizes both game-play quality and software engineering quality by using a simple but well-structured game loop, reusable widgets, and maintainable state management.

The core gameplay is based on memorizing and repeating a sequence of highlighted tiles. At the beginning of a round, the application displays a pattern of button activations. The player must then reproduce the same sequence by tapping the correct tiles in order. After each successful round, the system appends one additional random step to the sequence and slightly increases the speed, making the game progressively more challenging.

The project includes two main gameplay modes:

- **Focus Mode:** a 2×2 board intended for easier play and smoother onboarding.
- **Overdrive Mode:** a 3×3 board intended for advanced players and faster-paced gameplay.

Beyond the game loop itself, the application includes a point system, persistent high-score storage, streak tracking, mode selection, restart behavior, leaderboard functionality, and a dedicated game over overlay. The goal of the project is not only to deliver a functional game, but also to demonstrate proper Flutter application architecture, responsive UI design, local persistence, and user-centered interaction design.

2. Project Objectives

The proposed project aims to achieve the following objectives:

1. Build a complete Flutter mobile application with a clean and modular structure.
2. Implement a memory-based game loop that reacts correctly to user input and increasing difficulty.
3. Provide two difficulty modes through different board sizes and pacing.
4. Store high-score related information locally so that user progress persists between sessions.
5. Design a modern, visually engaging interface suitable for a mobile device.
6. Demonstrate practical use of Flutter widgets, layouts, timers, navigation, overlays, and local storage.

7. Add a leaderboard system that displays the highest scores and motivates users to improve their ranking.
8. Implement a streak system that rewards players for completing consecutive correct sequences.

3. Game Logic Overview

The internal game logic is centered around a hidden list of numbers, where each number corresponds to a tile index on the board.

- **Sequence generation:** The application initializes an empty sequence and adds a random tile index to start the first round.
- **Sequence playback:** The system highlights the tiles one by one using a timer-based loop so the player can memorize them.
- **User input handling:** The application listens for tile taps and compares each tap to the current expected value in the hidden sequence.
- **Round progression:** If the player completes the full sequence correctly, the score increases, the streak continues, one new step is appended, and playback speed becomes slightly faster.
- **Failure handling:** If the user taps the wrong tile, the round ends immediately and the game over overlay is displayed.
- **Persistence:** The best score, best streak, and leaderboard records are stored in local phone memory using persistent storage.

4. Additional Game Features

4.1. Leaderboard System

To make Neural Recall more competitive and engaging, the application will include a leaderboard system. The leaderboard will display the highest scores achieved by players, allowing users to compare their performance and feel motivated to improve their memory, reaction speed, and sequential recall.

The leaderboard can show important information such as the player's name, highest score, best streak, and selected game mode. Since the game contains both Focus Mode and Overdrive Mode, the leaderboard may also separate scores by mode so that players are compared fairly based on difficulty.

The main purpose of the leaderboard is to increase replay value. Instead of only playing once, users will have a reason to return to the game, beat their previous score,

and try to reach a higher rank. This feature also makes the application feel more complete and closer to a real mobile game.

Main functions of the leaderboard:

- Display the top player scores.
- Show the player's best score and best streak.
- Rank players based on performance.
- Separate leaderboard results by game mode if needed.
- Encourage users to replay the game and improve their results.

The leaderboard data can first be stored locally using `SharedPreferences`. If the project is extended in the future, it can be connected to an online database such as Firebase Firestore to support global rankings between different users.

4.2. Streak System

Another important feature in Neural Recall is the streak system. The streak represents how many correct sequences the player completes in a row without making a mistake. This feature rewards consistency, focus, and accuracy rather than only rewarding the final score.

Each time the player successfully repeats the full sequence, the streak value increases by one. If the player taps the wrong tile, the streak ends and the game over overlay is displayed. The best streak achieved by the user will be saved locally, allowing the player to track long-term progress even after closing the application.

The streak system can also be used to create bonus rewards. For example, when the player reaches certain streak milestones such as 5, 10, or 20 correct rounds, the game may give bonus points or display a special visual effect. This makes the gameplay more exciting and gives the player smaller goals during each session.

Main functions of the streak system:

- Track consecutive successful rounds.
- Reset the streak when the player makes a mistake.
- Save the best streak locally.
- Display the current streak during gameplay.
- Reward high streaks with bonus points or visual feedback.

This feature improves the game because it gives players another reason to keep playing. A player may not always reach the highest score, but they can still try to beat their best streak and improve their consistency.

5. Detailed Analysis Using Wireframes

This section analyzes the provided interface wireframes and explains the purpose of each screen.

5.1. Screen 1 — Main Menu

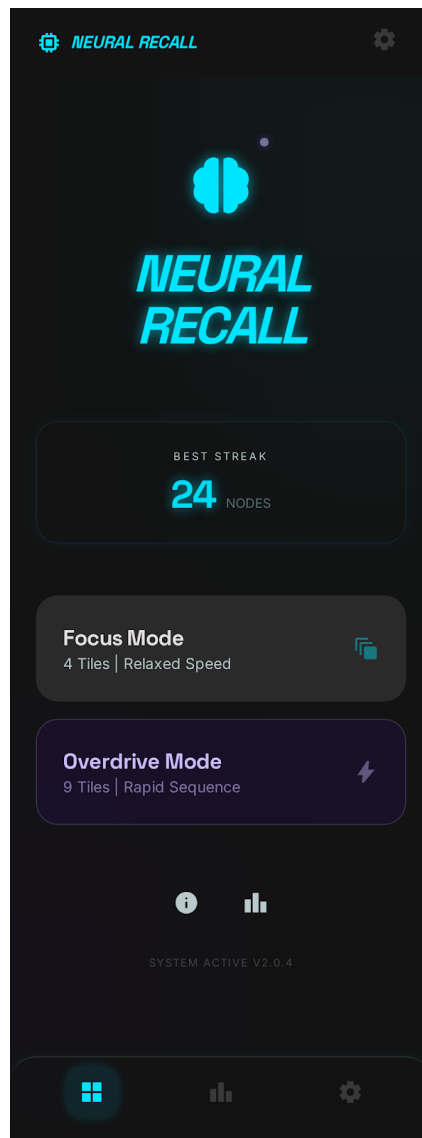


Figure 1: Main Menu Screen

The Main Menu acts as the application entry point. It presents the game identity, displays the player's Best Streak, and allows the user to choose between the available game modes. This screen is intentionally simple so that the player can immediately understand the purpose of the app and start playing with minimal friction.

Main functions of this screen:

- Display the application title and visual theme.

- Show the best streak achieved by the user.
- Allow mode selection between Focus Mode and Overdrive Mode.
- Provide bottom navigation and settings access.

Design interpretation:

- The layout prioritizes readability and quick access.
- Large cards make mode selection mobile-friendly.
- The dark background with glowing accents supports the futuristic identity of the app.

5.2. Screen 2 — Focus Mode Game Board (2×2)

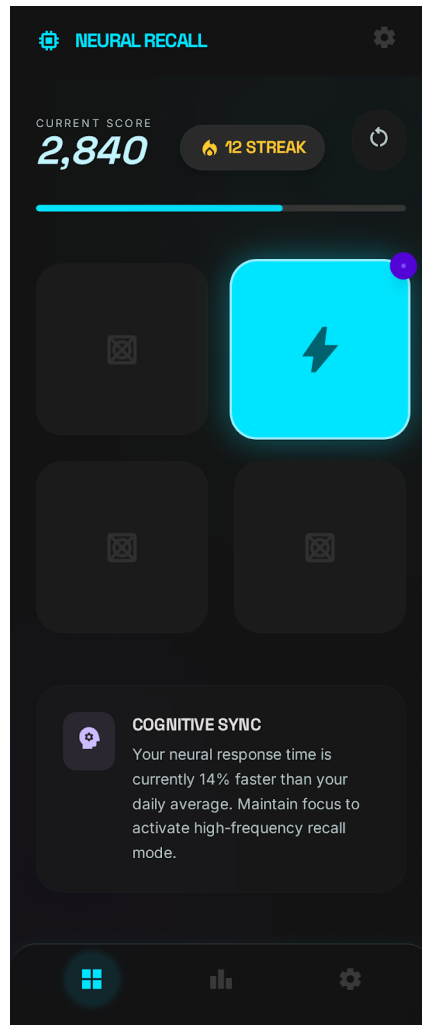


Figure 2: Focus Mode Gameplay Screen

This wireframe represents the easier gameplay mode with a 2×2 board. It includes the current score, the current streak, a progress bar, a reset control, and an informational card. The player taps the flashing tiles in the correct sequence.

Main functions of this screen:

- Display the current score and live streak.
- Present a 2×2 tappable board.
- Animate active tiles while replaying the sequence.
- Track round progress visually through the bar.
- Allow restarting the current game quickly.

Design interpretation:

- The simpler board lowers cognitive load for first-time or casual users.
- The score and streak are placed near the top to give immediate feedback.
- The info card can be used to motivate or guide the player during gameplay.

5.3. Screen 3 — Overdrive Mode Game Board (3×3)

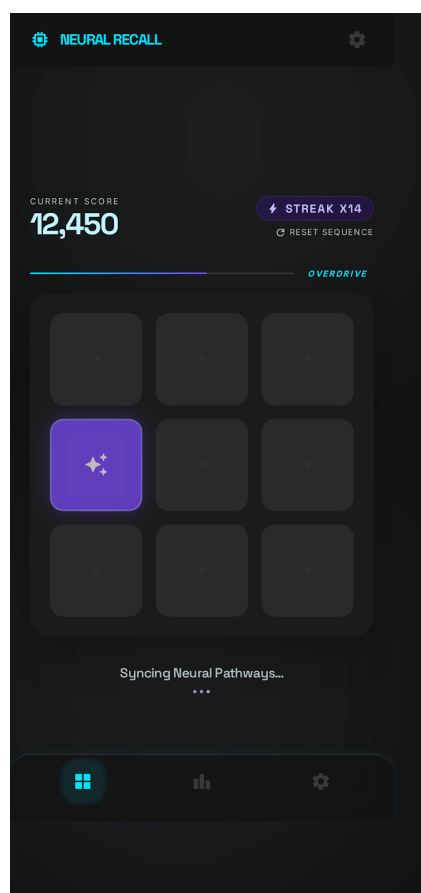


Figure 3: Overdrive Mode Gameplay Screen

This wireframe shows the advanced gameplay mode with a 3×3 board. The larger board size increases complexity, while the interface also emphasizes overdrive status and live sequence synchronization.

Main functions of this screen:

- Display the current score and a multiplied streak indicator.
- Present a 3×3 board for more advanced memory patterns.
- Support faster sequence playback than Focus Mode.
- Provide reset functionality.
- Inform the user about current sequence synchronization status.

Design interpretation:

- The larger board strongly differentiates the harder mode.
- Visual emphasis on Overdrive helps communicate increased difficulty.
- The spacing and tile layout are optimized for touch interaction even with more elements on screen.

5.4. Screen 4 — Game Over Overlay

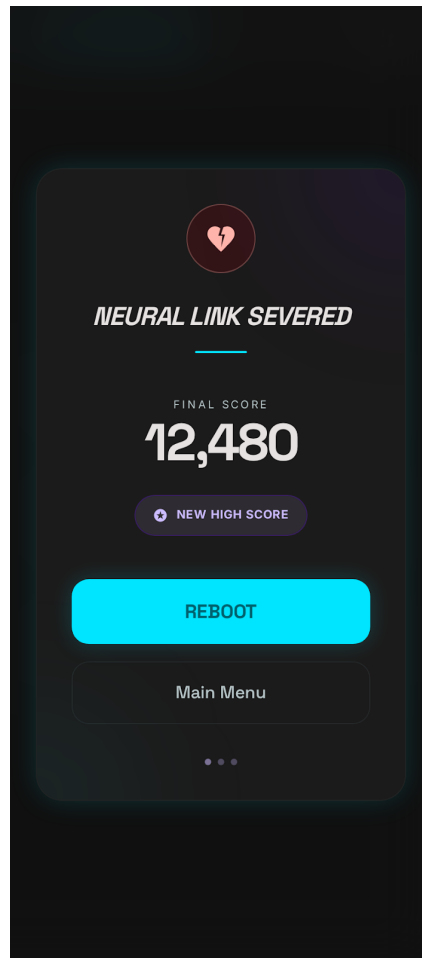


Figure 4: Game Over Overlay

The Game Over Overlay appears when the player makes an incorrect input and the sequence is broken. It presents the final score clearly and offers immediate recovery actions.

Main functions of this screen:

- Inform the player that the round has ended.
- Display the final score.
- Indicate whether a new high score was achieved.
- Offer a Reboot action to start again.
- Offer a return to the Main Menu.

Design interpretation:

- The modal format keeps the user in context and avoids abrupt navigation.

- The hierarchy of actions encourages starting another round quickly.
- Strong contrast between the overlay and background makes the end-state clear.

6. Flutter Widgets and Functional Implementation

This section explains the main Flutter widgets and structures that are expected to be used in the project.

6.1. Application-Level Widgets

Widget / Tool	Purpose in the Project
<code>MaterialApp</code>	Root widget for the application, theme setup, routing, and navigation structure.
<code>Scaffold</code>	Base layout for each main screen, including body content and bottom navigation placement.
<code>SafeArea</code>	Prevents content from overlapping with device notches and system UI areas.
<code>Navigator</code>	Handles movement between the main menu and game-play screens when needed.
<code>ThemeData</code>	Defines the overall visual identity, including dark theme colors and typography.
<code>StatefulWidget</code>	Used for screens that update dynamically based on timer progression, score, streak, and user interaction.
<code>SharedPreferences</code>	Stores persistent values such as high score, best streak, and leaderboard records in local phone memory.

6.2. Main Menu Screen Widgets

Widget	Expected Use
<code>Column</code>	Arranges title, best streak card, mode cards, and bottom controls vertically.
<code>Row</code>	Places icons and short sections side by side where necessary.
<code>Container</code>	Used for stylized cards, glowing sections, and background decoration.
<code>Text</code>	Displays title, labels, streak values, and mode descriptions.
<code>Card / Container</code>	Represents mode selection blocks for Focus Mode and Overdrive Mode.

<code>InkWell</code> <code>GestureDetector</code>	or	Detects taps on mode cards to start the selected game mode.
<code>Icon</code>		Used for game identity, settings, and bottom navigation symbols.
<code>BottomNavigationBar</code> or custom bottom bar		Provides quick navigation between app sections.

6.3. Focus Mode and Overdrive Mode Widgets

Widget	Expected Use
<code>GridView.builder</code>	Creates the interactive tile board dynamically for both 2×2 and 3×3 layouts.
<code>AnimatedContainer</code>	Animates tile color, glow, or shadow when a tile becomes active or selected.
<code>GestureDetector</code>	Captures player taps on each tile.
<code>Text</code>	Shows score, streak, labels, and mode indicators.
<code>LinearProgressIndicator</code> or custom progress bar	Displays progression through the current round or pacing information.
<code>Timer</code>	Controls the playback timing of the hidden sequence and the speed increase per round.
<code>setState()</code> or a state-management solution	Updates active tile states, score, streak, and round transitions.
<code>Stack</code>	Allows overlaying glowing effects, status labels, and floating indicators above the board.
<code>IconButton</code>	Used for settings and reset controls.

6.4. Leaderboard Widgets

Widget / Tool	Expected Use
<code>ListView.builder</code>	Displays leaderboard entries in a scrollable list.
Card or Container	Shows each player's ranking, name, score, streak, and game mode.
<code>Text</code>	Displays rank number, player name, score, mode, and streak information.
<code>SharedPreferences</code>	Saves and retrieves local leaderboard data.
<code>DropDownButton</code> or filter buttons	Allows the user to filter leaderboard results by Focus Mode or Overdrive Mode.

6.5. Game Over Overlay Widgets

Widget	Expected Use
<code>showDialog()</code> or <code>showGeneralDialog()</code>	Displays the game over overlay above the active game-play screen.
<code>AlertDialog</code> or custom <code>Dialog</code>	Holds the final score, high-score message, and next actions.
<code>ElevatedButton</code>	Used for the Reboot action.
<code>TextButton</code> or <code>OutlinedButton</code>	Used for returning to the Main Menu.
<code>Text</code>	Displays the game over title, score value, and high-score notice.
<code>Column</code>	Organizes the overlay elements vertically with clear hierarchy.

6.6. Functional Modules

Functionality	Implementation Approach
Sequence storage	A <code>List<int></code> stores the hidden button order for the current session.
Random step generation	<code>Random()</code> appends one valid tile index at the end of the list each round.
Input validation	The pressed tile index is compared to the expected element at the current pointer position.
Difficulty scaling	The delay duration in the timer is reduced gradually after successful rounds.
Score calculation	Points are awarded according to successful taps and possibly reaction speed.
Best streak persistence	Stored locally using <code>SharedPreferences</code> .
Leaderboard storage	The highest scores and best streaks are stored and displayed in ranked order.
Reset logic	Clears temporary game state, resets the pointer, and starts a new sequence.
Overlay trigger	Invoked when input validation fails.

7. System Architecture and Code Structure

To keep the code base clean and maintainable, the application is proposed to use a modular structure. A possible project structure is shown below:

```
lib/
|-- main.dart
|-- screens/
```

```

|   |-- main_menu_screen.dart
|   |-- focus_mode_screen.dart
|   |-- overdrive_mode_screen.dart
|   |-- leaderboard_screen.dart
|-- widgets/
|   |-- game_tile.dart
|   |-- score_panel.dart
|   |-- mode_card.dart
|   |-- leaderboard_card.dart
|   |-- game_over_dialog.dart
|-- services/
|   |-- storage_service.dart
|-- models/
|   |-- game_mode.dart
|   |-- game_state_model.dart
|   |-- leaderboard_entry.dart
|-- utils/
|   |-- constants.dart
|   |-- game_engine.dart

```

This structure separates UI, logic, storage, and reusable components. Such separation improves readability, supports testing, and makes the project easier to present academically.

8. Semester Deliverables and Progress Presentation Plan

The project requires 3–4 progress presentations during the semester. The following plan proposes 4 presentations, including the proposal presentation.

No.	Target Time	Main Goal	Features to Present
1	7 April	Project proposal presentation	Project concept, gameplay idea, wireframes, technical plan, Flutter widget mapping, semester roadmap, and acceptance criteria.
2	Week 4 after proposal	Core gameplay demo	Working navigation, main menu, Focus Mode board, sequence generation, tile flashing, user input validation, score update, and basic reset logic.

3	Week 8 after proposal	Extended feature demo	Overdrive Mode, difficulty scaling, improved animations, streak tracking, leaderboard system, local storage for best score and best streak, and game over overlay.
4	Final presentation / final delivery week	Final integrated system	Full application, bug fixes, polished UI, responsive behavior, persistent data, leaderboard results, final testing results, acceptance criteria checklist, and user acceptance test execution summary.

8.1. Detailed Semester Deliverables

Deliverable 1: Proposal Package

- Problem definition and project scope.
- Wireframe analysis.
- Technology plan and Flutter widget selection.
- Development roadmap.

Deliverable 2: Minimum Playable Prototype

- Main menu screen.
- Focus Mode playable board.
- Sequence playback and user tap validation.
- Initial score system.

Deliverable 3: Feature-Complete Beta

- Overdrive Mode board.
- Speed increase per round.
- Persistent high-score storage.
- Streak tracking.
- Leaderboard system.

- Game over overlay.
- Improved visuals and animations.

Deliverable 4: Final Release

- Full integration of all screens and features.
- Final testing and bug fixing.
- Acceptance validation and demonstration.
- Final submission package and presentation.

9. Final Acceptance Criteria

The project shall be accepted at final delivery if the following criteria are satisfied:

1. The application launches correctly on a Flutter-supported mobile environment.
2. The Main Menu is displayed correctly and allows the user to select a game mode.
3. Focus Mode starts a playable 2×2 memory game.
4. Overdrive Mode starts a playable 3×3 memory game.
5. The application generates and stores a valid hidden sequence of tile indices.
6. The sequence is replayed visually to the user before input is accepted.
7. User taps are validated in the correct order against the hidden sequence.
8. A wrong tap ends the round and displays the Game Over Overlay.
9. The final score is shown correctly at game over.
10. The application tracks the user's longest streak correctly.
11. The leaderboard displays saved high-score results correctly.
12. High score, best streak, and leaderboard data remain available after closing and reopening the app.
13. The reset functionality correctly restarts the current game session.
14. The UI remains usable, readable, and responsive on a mobile screen.
15. Navigation between the main menu and gameplay screens works without crash or logical error.
16. The application code is organized into reusable widgets and clear modules.

10. User Acceptance Test Cases

ID	Test Case	Expected Result	Acceptance Condition
UAT-1	Open the application	App opens without crash and displays Main Menu	Pass if the menu loads successfully
UAT-2	Select Focus Mode	Focus Mode gameplay screen opens with 2×2 board	Pass if board layout and controls appear correctly
UAT-3	Select Overdrive Mode	Overdrive gameplay screen opens with 3×3 board	Pass if board layout and controls appear correctly
UAT-4	Start a new round	Sequence is shown visually before input begins	Pass if user can clearly observe the sequence
UAT-5	Tap the correct sequence	Score increases and a new round begins with an extra step	Pass if progression occurs correctly
UAT-6	Tap an incorrect tile	Game Over Overlay appears immediately	Pass if the round ends and overlay is shown
UAT-7	Reach a new best score	High-score indicator appears on game over	Pass if the app identifies the new best result correctly
UAT-8	Restart using Reboot button	Current session resets and a new game starts	Pass if previous temporary game state is cleared
UAT-9	Close and reopen the app after setting a best score	Best score / best streak remains stored	Pass if data persists correctly
UAT-10	Use the app continuously for several rounds	No major lag, freeze, or state inconsistency occurs	Pass if gameplay remains stable
UAT-11	Open the leaderboard screen	The highest scores and streaks appear in ranked order	Pass if leaderboard data displays correctly
UAT-12	Reach a streak milestone	Streak value updates and bonus feedback appears if implemented	Pass if streak progression works correctly

11. Risk Assessment and Mitigation

Risk	Impact	Mitigation
Incorrect state updates during gameplay	Sequence, score, or tap validation may fail	Separate game logic from UI and test round flow incrementally
Timer synchronization issues	Playback may overlap with user input	Lock user input during playback and enable it only after sequence display ends
UI inconsistency across screens	Poor user experience and weak presentation quality	Reuse shared styles and components across all screens
Persistence bugs	High score may not save correctly	Test <code>SharedPreferences</code> early and verify saved values after restart
Leaderboard ranking errors	Scores may appear in the wrong order	Sort leaderboard entries before displaying them and test using sample data
Late integration of features	Final demo quality may be reduced	Follow milestone plan and validate each feature before moving forward

12. Conclusion

Neural Recall is a suitable mobile computing semester project because it combines interactive gameplay, progressive difficulty, local persistence, and a modern user interface in one manageable but technically meaningful Flutter application. The project demonstrates practical understanding of widgets, layouts, timers, local storage, event handling, overlays, and clean code organization.

The proposed implementation plan, wireframe analysis, deliverable schedule, and final acceptance criteria provide a clear path from proposal to final delivery. With the two gameplay modes, persistent score tracking, streak system, leaderboard functionality, and polished visual design, the project is expected to be both academically strong and engaging for users.